

Deep Learning Aided Sensor Fusion for Drift Reduced IMU Orientation Estimation

Rahul Verma
201629064

Supervisor: Dr Zhiqiang Zhang

A project presented for the degree of
Masters of Engineering in Electronic Engineering

School of Electrical and Electronic Engineering
University of Leeds

Abstract

Inertial Measurement Units (IMUs) are widely used in a variety of applications, such as Body Sensor Networks (BSNs) for orientation estimation. However the gyroscope suffers from drift due to sensor bias and noise that when integrated accumulates over time. This project investigates a deep learning-based, approach which aims to mitigate gyroscopic errors which can be integrated with sensor fusion techniques, to achieve more accurate orientation estimates. The proposed deep-learning architecture leverages both neural networks and a temporal history to learn complex and nonlinear error patterns in IMU data, exploring if it outperforms a standard Kalman Filter without learned corrections. The network outputs a correction for the incoming gyroscope sample, and the data used in training, testing and validating the model comes from simulations through MATLAB's Navigation Toolbox and from public datasets, such as the Berlin Robust Orientation Estimation Assessment Dataset (BROAD).

Acknowledgements

I would like to express my sincere gratitude to Professor. Zhiqiang Zhang for his continuous support and guidance throughout this project. Professor. Zhang's insights and advice during this project, in particular the explanation of quaternion algebra and in-depth explanation of filters, were invaluable in shaping the direction of my work. In addition, offering his personal copy of Body Sensor Networks to aid in my understanding was highly appreciated.

I would also like to acknowledge the authors and maintainers of the publicly available datasets of BROAD and RepoIMU. Their efforts in collecting and labelling high quality IMU data, and optical motion capture ground truth enabled the training, testing, and validation of the neural network. I am grateful for their contributions and cite them accordingly in this report.

Contents

Abstract	1
Acknowledgements	1
List of Figures	2
List of Tables	3
1 Introduction	4
1.1 Inertial Measurement Units (IMUs) and their applications	4
1.2 Problem Statement: IMU Drift and Its Impact	4
1.3 Research Question and Hypotheses	4
2 IMU Basics and Operational Principles	5
2.1 Gyroscope: Operations and Errors	5
2.1.1 Angular Rate Integration	5
2.1.2 Gyroscope Error Sources	5
2.2 Accelerometer: Operations and Errors	6
2.2.1 Accelerometer Error Effects	6
2.3 Magnetometer: Operations and Errors	7
2.3.1 Magnetometer Error Sources and Effects	7
3 Unaided Unscented Kalman Filter Evaluation	8
3.1 Unscented Kalman Filter (UKF) Overview	8
3.2 Datasets and Experimental Data	8
3.3 Filter Setup	8
3.3.1 Adaptive Gating for Disturbed Measurements	8
3.4 Scenarios and Error Metrics	9
3.4.1 Quaternion Sign Convention	9
3.4.2 Error Determination	9
3.5 Results	9
3.5.1 Undisturbed Slow Rotation	9
3.5.2 Undisturbed Fast Rotation	10
3.5.3 Disturbed Stationary Magnet	11
4 Deep Learning Architecture	13
4.1 Aims of the Deep Learning Model	13
4.2 Model Selection	13
4.2.1 Recurrent Neural Networks (RNNs)	13
4.2.2 Convolutional Neural Networks (CNNs)	14
4.2.3 Temporal Convolutional Networks (TCNs)	15
4.2.4 Summary and Choice	15
4.3 TCN Design	16
4.3.1 Normalisation	16
4.3.2 Activation Functions	17
4.3.3 Regularisation	17
4.3.4 Residual Connections/1x1 Convolution	18
5 Conclusion and Next Steps	19
5.1 Semester 2 Aims	19
5.1.1 Model Hyperparameters	19
5.1.2 Model Training	19
5.2 Summary of Progress	19
5.3 Challenges	19
5.4 Skills Developed	19

List of Figures

1	Bias Effect on Gyro [7]	6
2	Effect of ARW on Gyro Output [8]	6
3	Geodesic Angle Rotation Error for Undisturbed Slow Rotation	9
4	Geodesic Angle Rotation Error for Undisturbed Fast Rotation	10
5	Geodesic Angle Rotation Error for Disturbed Stationary Magnet	11
6	Comparison of Yaw between Ground Truth and UKF fusion for Disturbed Stationary Magnet	12
7	RNN structure [19]	13
8	1D CNN structure [25]	14
9	CNN structure with dilations [27]	15
10	Structure of residual block	16

List of Tables

1	UKF configuration	8
2	Summary of geodesic angle error θ for undisturbed slow rotation	10
3	Summary of geodesic angle error θ for undisturbed fast rotation	10
4	Summary of geodesic angle error θ for disturbed stationary magnet	11
5	Common normalisation methods.	16
6	Common activation functions.	17

1 Introduction

1.1 Inertial Measurement Units (IMUs) and their applications

IMUs are composed of multiple sensors which include a gyroscope, accelerometer, and occasionally a magnetometer. These three sensors measure the angular rate, specific force, and the local magnetic field vector respectively. These measurements can be used to estimate the orientation of an object through integration of the angular rate. This data acquisition is essential for several applications such as BSNs[1], robotics, and autonomous vehicles where these systems rely on high-rate orientation updates.

IMUs have emerged as a key technology due to their ability to work in a self-contained environment. In environments where external references of orientation are unavailable or unreliable, technologies such as IMUs are attractive. Filters, such as the Kalman filter (KF), allow the use of accelerometers and magnetometers to guide and correct state estimation, but these also suffer from failures like magnetic disturbances, and high linear accelerations. Due to these limitations, there is significant motivation to explore deep-learning methods that compensate for these conditions and errors.

1.2 Problem Statement: IMU Drift and Its Impact

IMUs have shown promise in determining the orientation of an object in motion. However, IMUs suffer from a limitation called drift. IMU drift is characterised by the accumulation of errors through the integration of the angular rate. The sources of these errors include constant bias, scale factor errors and others expanded in section 3.1.2. Errors are not exclusive to the gyroscope but also affect the accelerometer and magnetometer. Drift is also dependent on the type of IMU that is used. Lower cost/grade IMUs suffer from drift at a higher magnitude which results orientation inaccuracies much quicker compared to higher cost/grade IMUs. Errors then lead to inaccuracies in the orientation estimation of an object determined by the IMU.

Kianifar et al. explored using IMUs for orientation estimation in a clinical setting. They found that for rotation angles parallel to gravity, drift due to gyroscope bias cannot be compensated by the accelerometer.[2]. Even with multiple sensors, it is still challenging to find an accurate orientation estimation. Thus it is important to try and address the orientation problem by addressing gyroscopic drift.

Therefore, this project aims to address gyroscopic drift by using deep-learning methods to learn complex and non-linear nature of biases and errors.

1.3 Research Question and Hypotheses

The question this project aims to answer is: **Can Deep Learning be used to learn the drift pattern to get drift free orientation estimation?** While this is the overarching question, it can be broken down to the following:

- How effectively can deep learning learn the drift experienced?
- Can this model be generalised or will the model only apply to a single dataset?
- What are the advantages or disadvantages of using Deep Learning compared to traditional approaches?

The hypothesis that I state is that the by incorporating a deep-learning model, we will be able to see a percentage decrease in the orientation error over a period of time.

2 IMU Basics and Operational Principles

2.1 Gyroscope: Operations and Errors

The gyroscope is the main component that is used to determine the orientation of the object. It measures the angular rate in its orthogonol axes $\omega^x, \omega^y, \omega^z$ which is used to determine the object's orientation at a discrete time. The orientation is determined through the integration of angular rate. There are multiple different ways to represent the orientation of the object, these being, Euler angles, rotation matrices, and quaternions. Euler angles give rise to the singularity problem due to the loss of one degree-of-freedom whenever two axes of the rotations are parallel [3]. Rotation matrices are also used, however they are less concise as they are represented as 3x3 matrix, where 6 of these elements are redundant[3]. Therefore in this chapter, the quaternion representation is selected due to being more computationally efficient than the others [4].

2.1.1 Angular Rate Integration

We can define a quaternion that is represented by the angular rate, where ω is the real angular rate, $\omega_q(t)$ is the pure quaternion. Note: Both the attitude and updated quaternions should be normalised to ensure quaternions represent rotations. $q_k \leftarrow \frac{q_k}{\|q_k\|}$.

$$\omega_q(t) = [0, \omega_x(t), \omega_y(t), \omega_z(t)] \quad (1)$$

The orientation evolves according to [3]

$$\hat{q}_k = \hat{q}_{k-1} \otimes \Delta q_k \quad (2)$$

where Δq_k is [3]

$$\Delta q_k = \begin{bmatrix} \frac{\omega}{|\omega|} \sin \frac{|\omega|\Delta t}{2} \\ \cos \frac{|\omega|\Delta t}{2} \end{bmatrix} \quad (3)$$

This was all done by using an ideal ω_k , if were to model the angular rate from a gyroscope as [5]

$$\omega_{m,k} = \omega_k + b_k + n_k \quad (4)$$

Where $\omega_{m,k}$ is the measured angular rate, ω_k is the true angular rate, b_k is the bias term, and n_k is the noise term. The real quaternion orientation update is as follows where Δq_k becomes $\Delta q_{m,k}$.

$$\Delta q_{m,k} = \begin{bmatrix} \frac{\omega_{m,k}}{|\omega_{m,k}|} \sin \frac{|\omega_{m,k}|\Delta t}{2} \\ \cos \frac{|\omega_{m,k}|\Delta t}{2} \end{bmatrix} \quad (5)$$

and the update is now

$$\hat{q}_k = \hat{q}_{k-1} \otimes \Delta q_{m,k} \quad (6)$$

This shows that the inclusion of the bias and noise accumulates over samples as we move forward to the next k, q_{k-1} will incorporate the previous error terms. This results in an accumulated orientation error.

For more information about this derivation see Chapter 12[3].

2.1.2 Gyroscope Error Sources

Constant Bias

The bias of a gyroscope is the average output from the gyroscope when it is not undergoing any rotation [6]. This is measured in $^\circ/h$ and can be estimated by taking an average of the output. As this bias is constant, the drift it causes grows linearly with time. However, as discussed further in this section other error sources can make this difficult to determine. Figure 1 shows how constant bias changes the output.

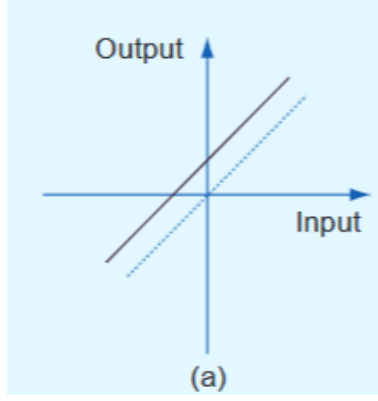


Figure 1: Bias Effect on Gyro [7]

White Noise / Angle Random Walk (ARW)

The gyroscope is also affected by some white noise [6]. This white noise sequence is zero-mean uncorrelated random variables between samples and across axes. When the gyroscope signal is integrated to obtain an angle, this white noise produces an ARW. The units of ARW is denoted by $^{\circ}/\sqrt{h}$, this shows that the deviation of angle error grows proportionally to $\sqrt{t}$. As ARW is due to random variables, it is classified as the 1σ of the orientation error. Analog devices shows this in the figure 2, where the ARW is $0.17^{\circ}/\sqrt{h}$.

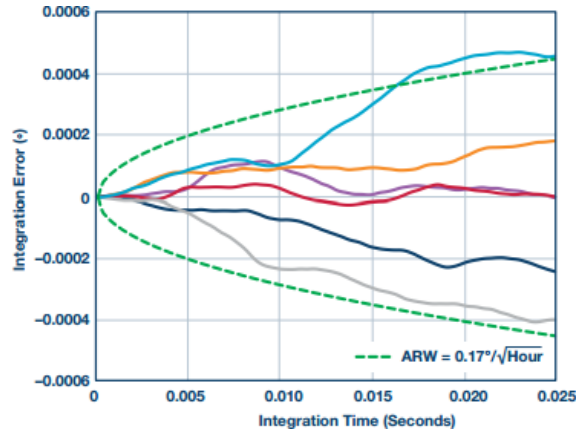


Figure 2: Effect of ARW on Gyro Output [8]

For more gyroscope error sources and effects, see [6].

2.2 Accelerometer: Operations and Errors

The accelerometer is another component used in determining an orientation estimate of an object. It measures the specific force along its orthogonal axes a^x, a^y, a^z . The main purpose of the accelerometer is to offer a gravity reference to the system such that, an accelerometer can provide drift-free inclination estimates [9]. However, the accelerometer, as stated above, does not only measure a gravity reference but also any linear accelerations that the object also experiences. Under conditions of high accelerations, the reference is unreliable and is only reliable for static or slowly moving objects [10]. Additionally, the accelerometer cannot detect rotations about the gravity vector, so it cannot provide a yaw/heading of the object. However, like the gyroscope, it also suffers from a variety of errors which causes instability in the output.

2.2.1 Accelerometer Error Effects

Accelerometers suffer from a very similar sources of errors to the gyroscope (section 3.1.2). Therefore, in this section we will discuss the implications of these errors for our orientation estimates.

Constant Bias

The effect of constant bias is that it causes an offset in the roll/pitch of the system. Unlike constant bias in the gyroscope, it does not grow linearly with time but is a constant offset irrespective of time elapsed.

High Linear Acceleration

As discussed above, the largest problem is high linear accelerations which corrupts the gravity reference. This is a dominant issue in high dynamic motions as the accelerometer will not be able to correct the orientation estimation through fusion, especially if gyroscopic drift is not addressed.

2.3 Magnetometer: Operations and Errors

Magnetometers measure the local magnetic field along its orthogonal axes m^x, m^y, m^z . The magnetometer can be used to determine the heading (yaw) of an object. Sensor fusion/filtering algorithms are dependent on sensing the local magnetic field to eliminate drift in the azimuth (yaw) portion of orientation estimates [11]. It does this by assuming that the measured local field vector provides a reference relative to Earth's field, however changes in the local field due to other field sources, can distort this measurement and gives rise to erroneous heading corrections.

2.3.1 Magnetometer Error Sources and Effects

Hard Iron/ Soft Iron

Hard Iron is a constant additive magnetic field applied to the sensor due to the interaction of magnetic fields [12]. This interaction between these sources leads to a constant vector being applied to the measurement.

Soft Iron are the effects generated by the interaction of an external magnetic field on any ferromagnetic materials near the sensor [12]. The resulting magnetic field depends on the magnetic field vector with respect to the orientation of the soft iron material [13].

3 Unaided Unscented Kalman Filter Evaluation

3.1 Unscented Kalman Filter (UKF) Overview

The UKF is an extension of the KF, it is an algorithm that is used in state estimation and can be used to fuse our gyroscope, accelerometer, and magnetometer to get an overall orientation estimate of our system. Unlike standard KF, the UKF can model the non-linear relationships in orientation estimation due to quaternion integration. Another alternative to the UKF is the extended Kalman filter (EKF), however, better performance tends to be achieved by the UKF [14]. Therefore, for this project the UKF was selected as the sensor fusion algorithm due to its tendency to yield better performance.

For an in-depth derivation regarding the UKF see Chapter 12.3.3 [3].

3.2 Datasets and Experimental Data

The dataset used for UKF evaluation has been from BROAD [15] whilst training, testing and, validation data will be from a range of sources including BROAD, OxIOD [16], and others to improve generalisation of the model across sensors, motion scenarios, and disturbances.

The reason why BROAD was used for the baseline is that it has many appealing qualities, these being isolated magnetometer and accelerometer disturbances, short and long motions, and rest phases. These properties allow us to estimate the noise of each sensor for process and measurement noise covariances, explored in section 3.3, from the rest segments and tests the filter under disturbed conditions.

3.3 Filter Setup

Table 1 below shows the configuration used for the UKF for each of the scenarios.

State	Process Noise (Q)	Measurement Noise (R)	Sampling Rate
$q_w, q_x, q_y, q_z,$ $\omega_{b,x}, \omega_{b,y}, \omega_{b,z}$	Gaussian noise determined by rest phases	Gaussian noise determined by rest phases	286 Hz

Table 1: UKF configuration

The state estimates the orientation that is determined by integrating gyroscope measurements where there is an assumed slowly varying additive bias applied to the gyroscope. The matrix Q handles the uncertainty the filter has in the process model (gyro integration). The measurement model assumes that accelerometer updates are the measured specific force that aligns with gravity (low linear acceleration), and the magnetometer is a reference of the local Earth's field. The matrix R is the uncertainty that the filter has in accelerometer and magnetometer updates and is adaptively weighted.

3.3.1 Adaptive Gating for Disturbed Measurements

The matrix R applied an adaptive scheme where disturbed accelerometer and magnetometer measurements were downweighted for correcting the orientation. The initial covariance was determined by taking a mean of all measurements for each sensor during the rest periods, and subtracting this mean, to get residuals from the sensors. These residuals were then computed as the covariance which is modelled as Gaussian noise and before each correction applied by the filter, a heuristic approach was taken by tuning arbitrary tolerances to accept or reject the sensor measurements. If measurements was deemed unreliable, then the matrix R would be multiplied by a scalar value to downweight updates during corrections, this scheme was selected as by outright rejecting measurements caused jerking orientation estimates.

The matrix Q , also took a similar approach where gyro bias was determined through subtracting the mean during rest phases and these residuals were computed as the covariance which was modelled to be Gaussian noise.

3.4 Scenarios and Error Metrics

The scenarios chosen to evaluate the performance of the filter were undisturbed slow rotation, undisturbed fast rotation, and disturbed stationary magnet [15]. The first scenario was chosen to see how the filter behaves under ideal conditions where accelerometer and magnetometer measurements were close to ideal. The second scenario for how high linear accelerations corrupt orientation estimates (inclination) given by the filter, and the final scenario for how magnetometer disturbances affect the yaw estimation of the filter.

3.4.1 Quaternion Sign Convention

When orientations are represented by unit quaternions it is important to note that

$$\mathbf{q} \equiv -\mathbf{q} \quad (7)$$

represents the same rotation.

Meaning that when determining the quaternion error between ground truth and estimation, we enforce a sign by checking the dot product between the two quaternions.

3.4.2 Error Determination

To determine the error between the two quaternions, we use the geodesic rotation angle between the two quaternions. This metric measures the minimal rotation needed to align the two quaternions.

This is shown in the equation

$$\theta = 2 \arccos\left(|\mathbf{q}_{gt}^\top \mathbf{q}_{est}|\right). \quad (8)$$

In section 3.5, θ is summarised by the mean (μ), standard deviation (σ), median, minimum, and the maximum.

3.5 Results

3.5.1 Undisturbed Slow Rotation

Undisturbed slow rotation refers to the test BROAD had done under an undisturbed magnetic field and a slow rotation so there is low linear acceleration. Figure 3 and Table 2 shows a summary of the geodesic angle error from the UKF.

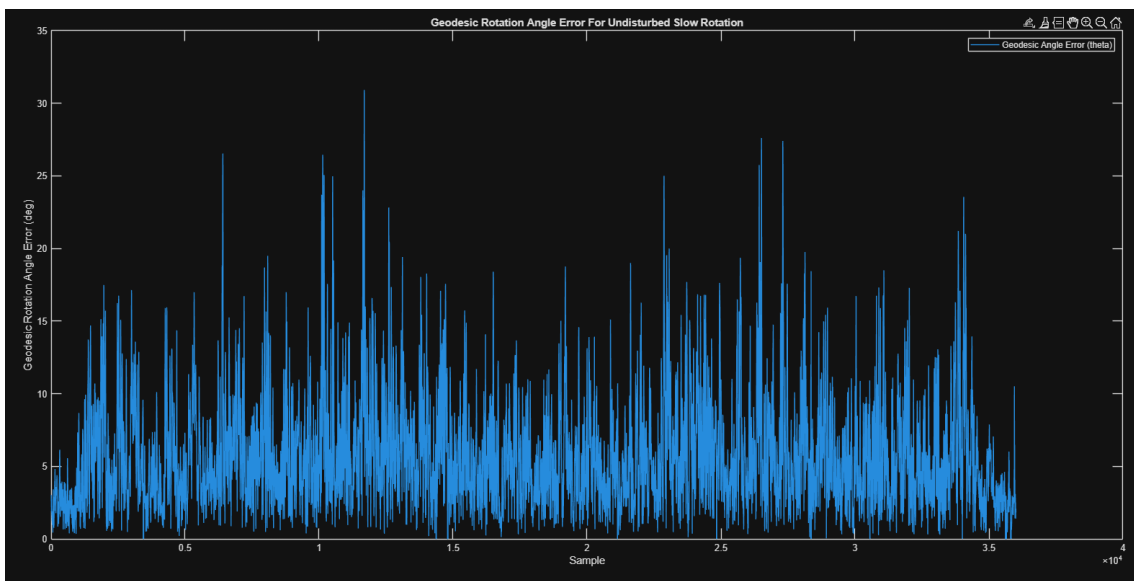


Figure 3: Geodesic Angle Rotation Error for Undisturbed Slow Rotation

Mean (μ)	Standard tion (σ)	Devia-	Median	Minimum	Maximum
5.8150	3.7535		4.9821	0	30.9128

Table 2: Summary of geodesic angle error θ for undisturbed slow rotation

As we can see from the graph and table, the UKF performed reasonably well in determining the orientation with a $\mu = 5.8150$, $\sigma = 3.7535$ median = 4.9821. This means that under undisturbed conditions of the magnetometer and accelerometer, it is a viable choice to use a UKF to determine the orientation. Additionally, with a max = 30.9128, even though it is quite high under these close to ideal conditions, the spikes were brief and the UKF was able to recover quite well to keep the mean low.

The error is still not very close to 0, under these conditions. This can be due to multiple factors including, gyroscopic drift caused by integration, the process and measurement noise not accurately modelling the sensors dynamics, and any misalignments in sensor axes.

3.5.2 Undisturbed Fast Rotation

Undisturbed fast rotation refers to the test BROAD had done under an undisturbed magnetic field and a fast rotation so there is high linear acceleration. Figure 4 and Table 3 shows a summary of the geodesic angle error from the UKF.

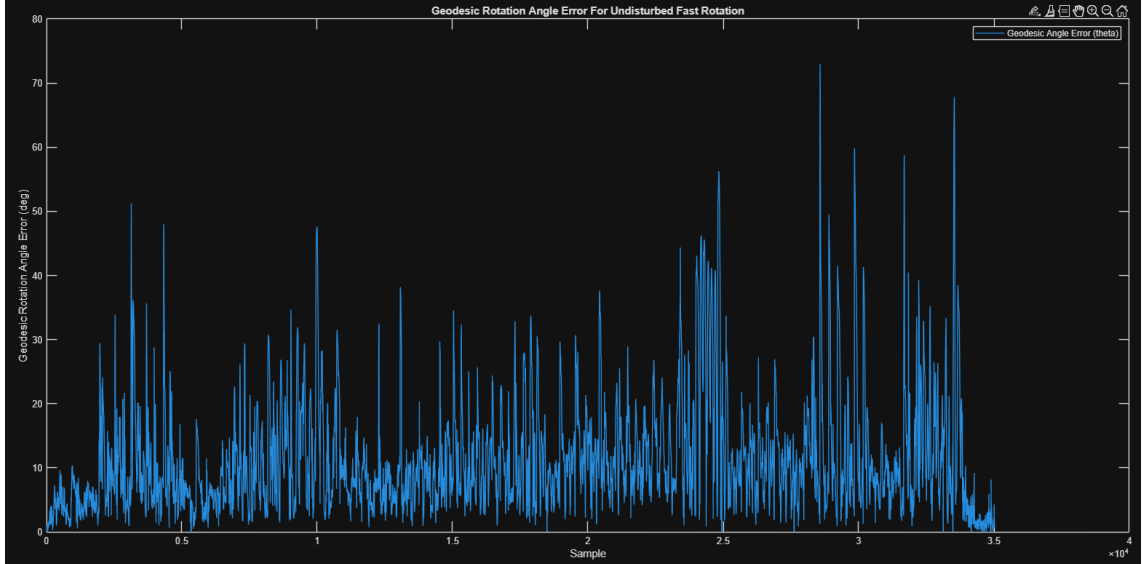


Figure 4: Geodesic Angle Rotation Error for Undisturbed Fast Rotation

Mean (μ)	Standard tion (σ)	Devia-	Median	Minimum	Maximum
11.4535	9.0682		9.0273	0	72.9758

Table 3: Summary of geodesic angle error θ for undisturbed fast rotation

As we can see from the graph and table, the UKF performed okay in determining the orientation with a $\mu = 11.4535$, $\sigma = 9.0682$ median = 9.0273. This means that under the disturbed conditions of the accelerometer, it can be seen that the UKF is a reasonable choice, however, there is still a lot of room for improvement on the orientation estimation. Furthermore, with a max = 72.9758, this is very high and can lead to severe inaccuracies if not addressed. Again with the previous test, the UKF was able to recover relatively well but this max would not be acceptable for an orientation of an object and can lead to severe consequences if not addressed.

3.5.3 Disturbed Stationary Magnet

Disturbed stationary magnet refers to the test BROAD had done under a disturbed magnetic field by placing a stationary magnet next to the IMU, and medium-speed rotation so there is medium linear acceleration. Figure 5 and Table 4 shows a summary of the geodesic angle error from the UKF.

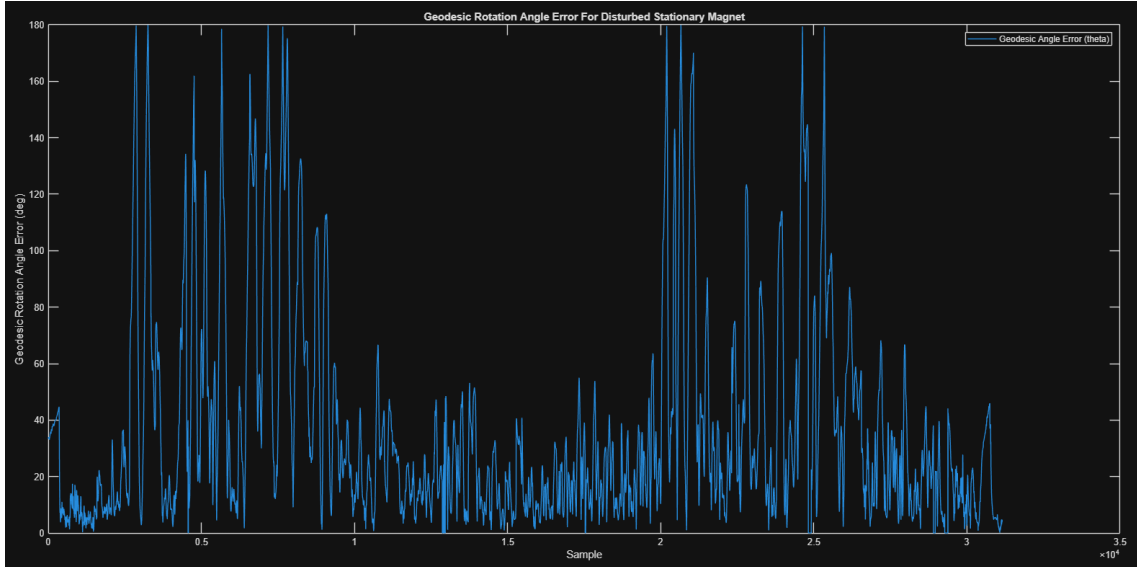


Figure 5: Geodesic Angle Rotation Error for Disturbed Stationary Magnet

Mean (μ)	Standard Deviation (σ)	Median	Minimum	Maximum
37.8035	38.8809	23.6325	0	179.9404

Table 4: Summary of geodesic angle error θ for disturbed stationary magnet

As we can see from the graph and table, the UKF performed terribly in determining the orientation with a $\mu = 37.8035$, $\sigma = 38.8809$ median = 23.6325. This means that under a magnetic disturbance, a UKF alone cannot be used for accurate orientation estimates even with adaptive gating for the magnetometer. As the gating for the magnetometer would have likely downweighted the magnetic reference, the accumulation of errors through integration would have also played a large role in the inaccuracy of the orientation.

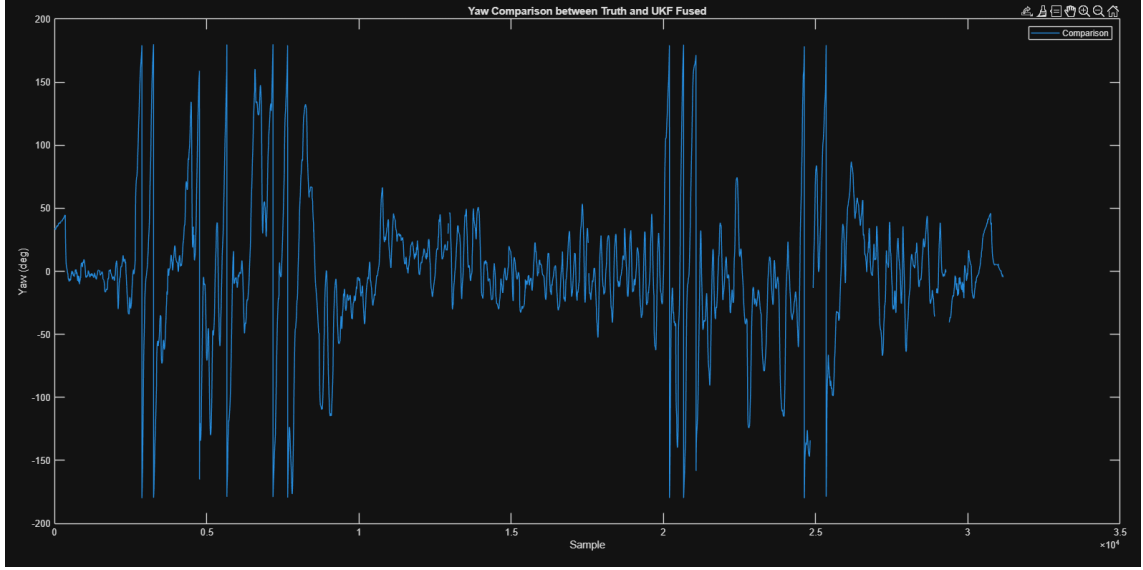


Figure 6: Comparison of Yaw between Ground Truth and UKF fusion for Disturbed Stationary Magnet

This can be further shown by looking at the comparison of the yaw between ground truth and fusion. Looking at Figure 6, we can see that the magnetic disturbance causes wide variations of the yaw throughout the test. This further supports the conclusion that without addressing gyroscopic drift through deep-learning then accurate orientation estimates from an IMU is unlikely.

4 Deep Learning Architecture

4.1 Aims of the Deep Learning Model

The motivation for learning $\Delta\omega_t = [\Delta\omega_x, \Delta\omega_y, \Delta\omega_z]^\top$ comes from the fact that while sensor fusion techniques of downweighting or skipping of the accelerometer and magnetometer when they are unreliable, drift cannot be mitigated due to the accumulation of gyroscopic bias and noise [17][18].

4.2 Model Selection

4.2.1 Recurrent Neural Networks (RNNs)

The first type of model that was considered is a RNN. RNNs stem from an idea of recurrence, that is that the output depends on some input at time t and an internal state of the model that holds information about previous outputs of the model. The general idea can be described by the equation

$$\hat{y} = f_{model}(x_t, h_{t-1}) \quad (9)$$

where $\hat{y}$ is the output, f_{model} , is a function that describes the model, x_t is the input at time t , and h_{t-1} is a vector state that describes the internal state of the model at the previous time step. Expanding on this, the state of the model is updated as follows

$$h_t = f_w(x_t, h_{t-1}) \quad (10)$$

where f_w is a function with weights w . This expands to the equations

$$\hat{y} = W_{hy}^T h_t \quad (11)$$

$$h_t = \phi(W_{hh}^T h_{t-1} + W_{xh}^T x_t) \quad (12)$$

where $W_{hh}^T, W_{hy}^T, W_{xh}^T$ are a set of weight matrices that are reused for every time step, and ϕ represents an arbitrary activation function.

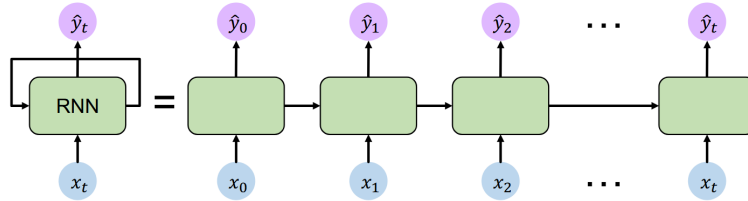


Figure 7: RNN structure [19]

Figure 3 shows the unrolled flow of the RNN, where the arrows between each block represents the passing of h_t , from one input to the next. The point of the recurrence state h_t , is to show how the output of the model depends on context from previous inputs plus h_t . This is especially relevant in our case, as described in section 3.1.1, the accumulation of gyroscopic errors comes from the integration of the angular rate. As these errors accumulate, the current error depends on the history of the gyroscope measurements.

There are many problems with RNNs however, the biggest being the problem of exploding/vanishing gradients. RNNs are trained via backpropagation through time, which propagates gradients across timesteps. For long sequences, computing the gradient with respect to h_0 involves many factors of repeated multiplication of weight matrices across time. This can lead to many values of the computation being greater than 1, which explodes gradients leading to unstable training. In the case, where many values are less than 1, this causes vanishing gradients where learning long-range dependencies becomes difficult. Additionally, as the hidden state h_t is propagated sequentially through time, when performing backpropagation, training

cannot be parallelised over timesteps. This results in a high training times compared to convolutional models.

Updated versions of the RNN have been proposed to solve the problem of vanishing and exploding gradients. These being Long Short Term Memory (LSTM) and Gated Recurrent Units (GRUs). These methods exploit the use of a cell state that does not need to do perform repeated multiplication of weight matrices, however they are still subject to long training times and the gradient problem is not fully solved but mitigated [20][21].

4.2.2 Convolutional Neural Networks (CNNs)

The next type of model considered was the CNN, even though typically it is used in computer vision tasks; it has seen wide use in research relating to drift reduction [22][23]. This model is based on the idea of convolving input data with a learnable filter to extract local features in the input data. The filter can be thought of like a window that slides over the data, and each filter has a defined set of weights in which it multiplies with input data. This convolution operation can be described by the equation below:

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t + k - \lfloor K/2 \rfloor] \quad (13)$$

$$y_f[t] = \phi(z_f[t]) \quad (14)$$

where $y_f[t]$, is the output of filter f , b_f is a bias, C_{in} is the input channels (for our case C_{in} would be 3 for $\omega_x, \omega_y, \omega_z$), K is the kernel/filter width, $w_{f,c}$ is the weights of the filter for a specific channel, x_c is the input data of that channel, and ϕ is an arbitrary non-linear activation function (typically ReLU).

For example if $K = 3$, the convolution will occur of the timesteps $x[t - 1], x[t], x[t + 1]$, this is how the convolution extracts local features in the input data.

Typically, these CNNs are made up of layers, where in each layer the number of filters grow e.g. $L1 = 16, L2 = 32, L3 = 64$ etc. These additional filters in later layers allow the model to learn more complex feature sets based on the previous layers' filter. Zeiler et al. [24] shows this in their figure 2 where their convolutional network is used for image classification.

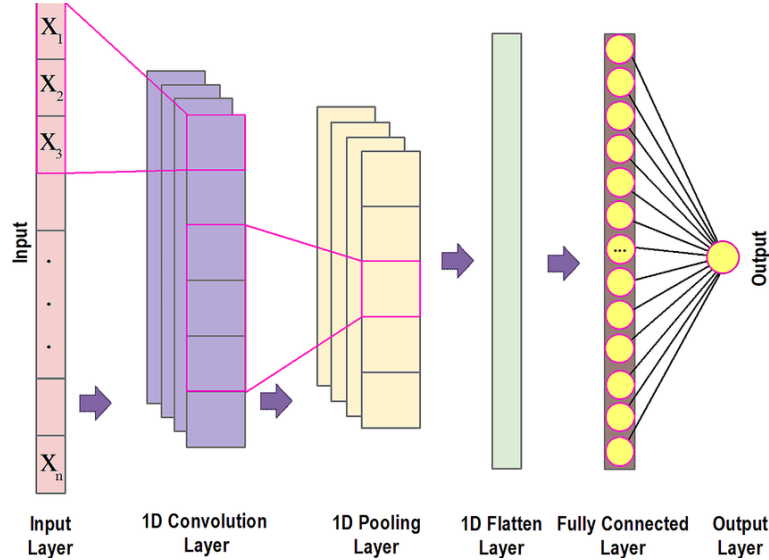


Figure 8: 1D CNN structure [25]

CNNs are more attractive than RNNs due to its ability to be efficiently trained by using parallelisation [26]. This leads to the ability to test multiple different iterations of a model where parameters and loss functions may be changed to explore what effects they may have (section 3.3). CNNs also do not suffer

from vanishing/exploding gradients as much and are seen to be more stable during training due to no backpropagation through time.

However, unlike RNNs, baseline CNNs cannot produce outputs due to significant temporal information. It can extract local features and in 1D convolutions it has some ability to traverse backwards through time but not to any significant degree. As seen in section 3.1.1, the accumulation of drift is due to the integration of the angular rate as we move forward in time.

4.2.3 Temporal Convolutional Networks (TCNs)

TCNs are an extension of the baseline CNN and include two main differences in their implementation as shown by Bai et al.[27]. These differences are the use of dilations that grow the receptive field allowing the network to look back further in time and learn long-term information. Furthermore, causal convolutions meaning, that the convolution operation cannot look forward in time. To explain causality, we can look back at equation 12 and modify this so

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t-k] \quad (15)$$

$$y_f[t] = \phi(z_f[t]) \quad (16)$$

The difference is that the k is subtracted from $x[t]$. Causality is enforced here as at time t , we cannot access future information i.e. $x[t+1], x[t+2] \dots$ etc.

Dilations allow the model to look further back in time without the need of increasing parameters for the same K and C_{in} used in convolutions. This is best represented in the figure 5.

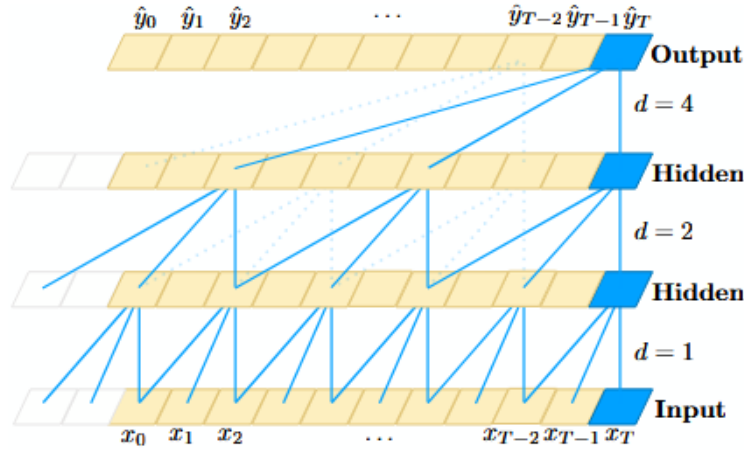


Figure 9: CNN structure with dilations [27]

As we can see in the diagram, as the dilations increase layer by layer, the output to the next layer looks further back into the history of $x[t]$. This can be modelled by the equation

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t-dk] \quad (17)$$

where d is the dilation factor of the layer.

4.2.4 Summary and Choice

In summary, the choice has been to select the TCN. This is because it can look back in history and model long-term dependencies between data, something baseline CNNs cannot do. Additionally, like a CNN it is quicker and more stable in training unlike an RNN. However, the TCN does have its problems, these

being extra data storage during its operation and domain transfer. Unlike RNNs which only have store $h(t)$, TCNs needs to take in a raw sequence upto its effective history length [27]. Domain transfer refers to transferring a model that has been trained in one domain (i.e. small receptive field) and applied to another domain (i.e. large receptive field), where the TCN may perform poorly due to this change in domain.

4.3 TCN Design

The main design of the TCN comes from Bai et al. [27]. Figure 6 demonstrates the architecture of the residual block that has been implemented in the first semester.

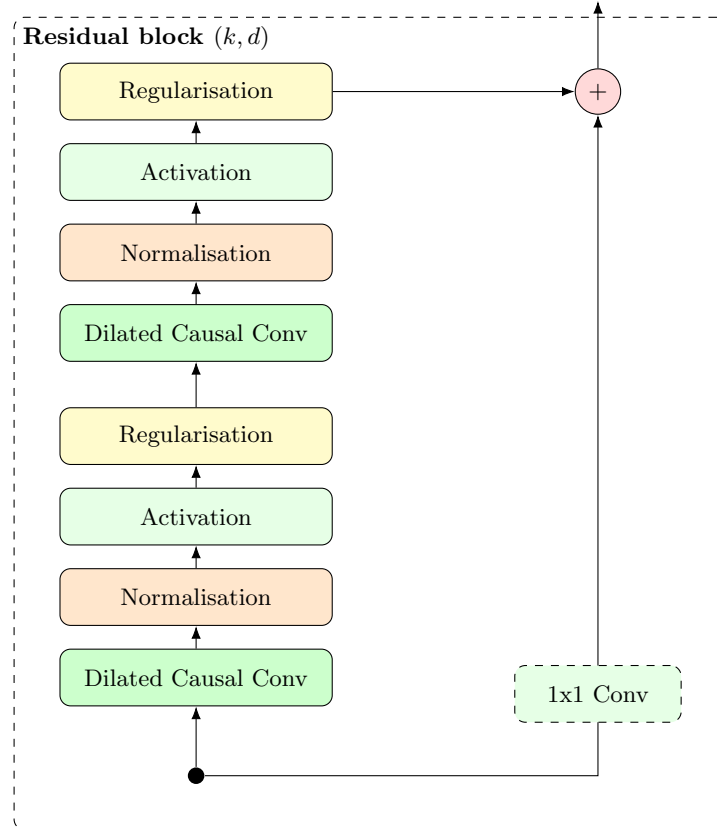


Figure 10: Structure of residual block

The dialted causal convolution is the same as discussed in section 3.2.3.

4.3.1 Normalisation

Normalisation is a technique used in deep learning to ensure training stability, while also reducing training times by allowing the use of higher learning rates. The aim of normalisation is to address internal covariate shift, the change in the distrubution of each layer’s inputs during training, as parameters of the previous layers changes [28]. This can be done by the following techniques.

Method	Normalises	Pros	Cons
BatchNorm	Activations (per channel)	Fast and adds some regularisation	small batch/streaming issues
LayerNorm	Activations (per token/sample)	Batch-size independent	sometimes worse performance in CNNs
GroupNorm	Activations (channel groups)	Works well with small batches	Number of groups need to be selected
WeightNorm	Weights (reparam)	Simple no batches	less regularisation than BatchNorm

Table 5: Common normalisation methods.

From the table above, in training BatchNorm, GroupNorm, and WeightNorm will be tested to see which technique yields the best model accuracy to model training stability. The reason why these have been selected is that BatchNorm has seen wide use in CNN models for drift reduction [22][23]. WeightNorm as the original TCN implementation uses it to avoid errors with small batch sizes [27]. GroupNorm as it is a middle ground where it works well with small batch sizes compared to BatchNorm but it may introduce more complexity with groups needing to be selected and as such will be tested last to see if the other techniques give an acceptable tradeoff.

4.3.2 Activation Functions

Activation functions are functions that are used in deep learning to add non-linearity to our output. This is because seen in equations 9,11,13,15, the relationship between weights and input data is all linear. The table below demonstrates common activation functions used.

Activation	Definition	Output range	When used	Pros	Cons
ReLU	$\text{ReLU}(x) = \max(0, x)$	$[0, \infty)$	CNN baseline	Fast activations	ReLU can result in 0 output (dying ReLU)
Leaky ReLU	$\max(\alpha x, x), \alpha \in (0, 1)$	$(-\infty, \infty)$	When ReLU units die	Reduces dying ReLU	extra hyperparameter α
GELU	$x \Phi(x)$ (Gaussian CDF)	$(-\infty, \infty)$	Transformers, deep networks	Smooth and good performance	slower than ReLU
Swish	$x \sigma(\beta x)$	$(-\infty, \infty)$	Transformers /CNNs	Smooth, often improves accuracy	slower than ReLU

Table 6: Common activation functions.

From the table above, the motivation for ReLU is that it is commonly used as the baseline for CNNs and is also used by Bai et al. in their TCN implementation. However, they can be prone to overfitting [22] or dying ReLU can occur. If ReLU is prone to dying ReLU, then leaky ReLU will be used and tested against the other functions. GELU and Swish are known to be smooth implementations of ReLU, where it is able to take values where $x < 0$ and can have slight performance improvements [29] but are computationally more expensive and performance is model dependent. For implementation, ReLU will be considered first for its computational efficiency, however if problems arise the other activation functions will be used.

4.3.3 Regularisation

Regularisation is a technique used in deep learning models to help models avoid overfitting to training data and allows the model to be more generalisable to unseen data. Overfitting is when the model learns its training data too precisely, so it performs well to training data but less than adequate to unseen data.

The technique that will be implemented first is dropout, where dropout randomly zeroes out all activations within the model for an input in training data. If the model is still prone to overfitting, more advanced regularisation techniques may be employed like weight decay which penalises large weights within kernels to allow for smoother functions.

4.3.4 Residual Connections/1x1 Convolution

The right branch (figure 6) is essential in training very deep neural networks where networks may not be able to converge due to exploding/vanishing gradients. This branch is called a residual connection, where the equation below shows its operation

$$y = \mathcal{F}(x, W_i) + Wx \quad (18)$$

where y is the output, $\mathcal{F}(x, W_i)$ is the residual mapping to be learned, W , is a projection applied to x (1x1 convolution), and x is our input[30].

Residual connections adds inputs of a block to the output of the block, so when learning the model learns the relationship of $\mathcal{F}(x, W_i) = y - Wx$, which is easier to learn than the full mapping directly. The 1x1 convolution is included as the model goes deeper, the number of channels increases to capture more complex features as each channel has their own set of filter/kernels. The addition between $\mathcal{F}(x, W_i)$ and x must be of two tensors of the same size so W allows the addition to be possible.

5 Conclusion and Next Steps

5.1 Semester 2 Aims

With an understanding of all IMU operations, a baseline of how the UKF is able to fuse sensor measurements and its performance, and with the residual block of the TCN implemented, semester two is on track for the completion of this project. The main core of the model is the residual block and from here the rest of the work is covered in the next subsections.

5.1.1 Model Hyperparameters

The main thing to do is to determine multiple of the model hyperparameters such as, the filter size, receptive field, dilation factor, and the number of channels per residual block. For dilation factor, it is a common method to follow is $d = 2^i$ or $d = 4^i$ where i , is the layer number as seen by [22]. This is because by increasing the dilation factor the model can see further back in time and use this information to more accurately predict the accumulation of errors. However, as this is not a standard rule a wide variety of dilation factors will be used to test model performance.

5.1.2 Model Training

5.2 Summary of Progress

During Semester One, the project progressed from initial understanding of the challenges faced in orientation estimation, to exploration and designing of deep learning models. Much of the work conducted had been research using academic literature and online lectures. With foundational knowledge gained in all areas of the project, it has ensured that Semester Two can be focused on the completion deep learning model to address the questions posed in section 1.3. Furthermore, the exploration of how the baseline UKF performed also gave me vital understanding in how the model should attempt to learn gyroscopic drift.

5.3 Challenges

The primary challenge encountered during this semester stemmed from having no prior knowledge of IMU operations and deep learning. This led to extensive research needed for the completion of the project, which slowed down the implementation of the model. However, now having concluded research I have been able to make informed decisions behind the selection of model, optimisation through normalisation, and measurements that accurately assess the performance of the model.

5.4 Skills Developed

The main skill developed has been learning how to assess and evaluate literature and ascertain the correct conclusions to be able to be applied to this project. My ability to use the PyTorch library and Python skills have developed to a significant degree in which modelling, measurements, and analysis will be conducted within Python. In addition, my technical writing capabilities have also progressed especially compared to my third-year project.

References

- [1] A. Filipposchi, N. Schmitz, M. Miezal, G. Bleser, E. Ruffaldi, and D. Stricker, “Survey of motion tracking methods based on inertial sensors: A focus on upper limb human motion,” *Sensors*, vol. 17, p. 1257, Jun. 2017. DOI: 10.3390/s17061257.
- [2] R. Kianifar, V. Joukov, A. Lee, S. Raina, and D. Kulić, “Inertial measurement unit-based pose estimation: Analyzing and reducing sensitivity to sensor placement and body measures,” *Journal of Rehabilitation and Assistive Technologies Engineering*, vol. 6, p. 205 566 831 881 345, Jan. 2019. DOI: 10.1177/2055668318813455. [Online]. Available: <https://journals.sagepub.com/doi/full/10.1177/2055668318813455>.
- [3] G.-Z. Yang, *Body Sensor Networks*. London Springer, 2014.
- [4] Z.-Q. Zhang and J.-K. Wu, “A novel hierarchical information fusion method for three-dimensional upper limb motion estimation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 60, pp. 3709–3719, Nov. 2011. DOI: 10.1109/tim.2011.2135070. (visited on 11/27/2021).
- [5] N. Trawny and S. Roumeliotis, *Indirect kalman filter for 3d attitude estimation*, 2005. [Online]. Available: <https://mars.cs.umn.edu/tr/reports/Trawny05b.pdf> (visited on 01/09/2026).
- [6] O. J. Woodman, “Number 696 an introduction to inertial navigation,” *An introduction to inertial navigation*, 2007. [Online]. Available: <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-696.pdf>.
- [7] “How good is your gyro [ask the experts],” *IEEE Control Systems*, vol. 30, pp. 12–86, Feb. 2010. DOI: 10.1109/mcs.2009.935122. (visited on 08/31/2021).
- [8] M. Looney, *Designing for low noise feedback control with mems gyroscopes*, Analog.com, Mar. 2016. [Online]. Available: <https://www.analog.com/en/resources/technical-articles/low-noise-feedback-control-mems-gyroscopes.html> (visited on 01/09/2026).
- [9] A. M. Sabatini, “Estimating three-dimensional orientation of human body parts by inertial/magnetic sensing,” *Sensors*, vol. 11, pp. 1489–1525, Jan. 2011. DOI: 10.3390/s110201489.
- [10] *Detection of static and dynamic activities using uniaxial accelerometers — ieee journals & magazine — ieee xplore*, ieeexplore.ieee.org. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=547939>.
- [11] E. Bachmann, X. Yun, and C. W. Peterson, “An investigation of the effects of magnetic variations on inertial/magnetic orientation sensors,” Jan. 2004. DOI: 10.1109/robot.2004.1307974. (visited on 08/02/2023).
- [12] J. Luiz, G. H. Elkaim, C. Silvestre, P. Oliveira, and B. Cardeira, “Geometric approach to strapdown magnetometer calibration in sensor frame,” vol. 47, pp. 1293–1306, Apr. 2011. DOI: 10.1109/taes.2011.5751259.
- [13] *Calibration of a magnetometer in combination with inertial sensors*, [Ieee.org](http://ieeexplore.ieee.org), 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/6289882> (visited on 01/10/2026).
- [14] E. Kraft, “A quaternion-based unscented kalman filter for orientation tracking,” Jan. 2003. DOI: 10.1109/icif.2003.177425.
- [15] D. Laidig, M. Caruso, A. Cereatti, and T. Seel, “Broad—a benchmark for robust inertial orientation estimation,” *Data*, vol. 6, p. 72, Jun. 2021. DOI: 10.3390/data6070072.
- [16] *Dataset and methods for deep inertial odometry*, [Ox.ac.uk](http://deepio.cs.ox.ac.uk/), 2019. [Online]. Available: <http://deepio.cs.ox.ac.uk/>.
- [17] Z.-Q. Zhang, X.-L. Meng, and J.-K. Wu, “Quaternion-based kalman filter with vector selection for accurate orientation tracking,” *IEEE Transactions on Instrumentation and Measurement*, vol. 61, pp. 2817–2824, Oct. 2012. DOI: 10.1109/tim.2012.2196397. (visited on 09/05/2020).
- [18] F. Bo, J. Li, W. Wang, and K. Zhou, “Robust attitude and heading estimation under dynamic motion and magnetic disturbance,” *Micromachines*, vol. 14, pp. 1070–1070, May 2023. DOI: 10.3390/mi14051070. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10221362/>.
- [19] A. Soleimany, *Deep sequence modeling mit 6.s191*, 2019. [Online]. Available: https://introtodeeplearning.com/2019/materials/2019_6S191_L2.pdf (visited on 01/15/2026).
- [20] S. Kanai, Y. Fujiwara, and S. Iwamura, *Preventing gradient explosions in gated recurrent units*. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/file/f2fc990265c712c49d51a18a32b39f0/Paper.pdf (visited on 01/15/2026).

- [21] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” *ResearchGate*, Nov. 2012. [Online]. Available: https://www.researchgate.net/publication/233730646_On_the_difficulty_of_training_Recurrent_Neural_Networks.
- [22] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising imu gyroscopes with deep learning for open-loop attitude estimation,” *IEEE Robotics and Automation Letters*, pp. 1–1, 2020. DOI: 10.1109/1ra.2020.3003256. (visited on 06/29/2022).
- [23] H. Chen, P. Aggarwal, T. M. Taha, and V. P. Chodavarapu, *Improving inertial sensor by reducing errors using deep learning methodology*, IEEE Xplore, Jul. 2018. DOI: 10.1109/NAECON.2018.8556718. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8556718>.
- [24] M. Zeiler and R. Fergus, *Visualizing and understanding convolutional networks*, Nov. 2013. [Online]. Available: <https://arxiv.org/pdf/1311.2901>.
- [25] P. V. Hoa, N. A. Binh, P. V. Hong, *et al.*, “One-dimensional deep learning driven geospatial analysis for flash flood susceptibility mapping: A case study in north central vietnam,” *Earth Science Informatics*, vol. 17, pp. 4419–4440, Jul. 2024. DOI: 10.1007/s12145-024-01285-8. (visited on 06/12/2025).
- [26] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Communications of the ACM*, vol. 60, pp. 84–90, May 2012.
- [27] S. Bai, J. Zico Kolter, and V. Koltun, *An empirical evaluation of generic convolutional and recurrent networks for sequence modeling*, Apr. 2018. [Online]. Available: <https://arxiv.org/pdf/1803.01271>.
- [28] S. Ioffe and C. Szegedy, *Batch normalization: Accelerating deep network training by reducing internal covariate shift*, 2015. [Online]. Available: <https://proceedings.mlr.press/v37/ioffe15.pdf>.
- [29] P. Ramachandran, B. Zoph, and Q. V. Le, *Searching for activation functions*, arXiv.org, 2017. [Online]. Available: <https://arxiv.org/abs/1710.05941>.
- [30] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, Jun. 2016. DOI: 10.1109/cvpr.2016.90. [Online]. Available: <https://ieeexplore.ieee.org/document/7780459>.